

## 13. Airbnb (Listings + Search + Booking)

---

**Interviewer:** Let's design Airbnb. Hosts list properties — photos, amenities, an availability calendar. Guests drag a map around, pick check-in/check-out dates, apply filters, and see a ranked list of places that are actually free for those dates. Then they book one. Where do you start?

**Candidate:** This is a search-heavy system with a small, strict transaction hiding at the bottom of it. The hard part isn't "how do I save a booking" — it's "how do I find, in under 300ms, every listing inside this map rectangle that is *available* for these five nights and matches four filters, out of tens of millions of listings." Let me clarify first.

---

### 1. Clarifying the requirements

---

**Candidate:** 1. Search is by **map viewport** ("search as I drag") and by city/text — right? 2. Availability — **per calendar night** per listing, so a 5-night stay can be free on nights 1–3 and blocked on night 4? 3. How stale can search results be — if a listing was booked 10 seconds ago, is a briefly stale "available" in results acceptable? 4. Is the booking transaction itself (payment, double-booking prevention) already solved elsewhere, or should I design it here too?

**Interviewer:** Yes to map-viewport plus city/text search. Availability is per night. Small staleness in search is fine — a few seconds. And assume booking's own correctness (payments, double-booking) is a solved subsystem elsewhere — focus on **search at scale** and the handoff to booking, not re-deriving the whole booking lock.

**Candidate:** Then requirements:

**Functional** - Host: create/edit listing (title, photos, amenities, geo-coordinates, price, per-night availability calendar). - Guest: `search(map bounds OR city, check-in, check-out, filters)` → ranked, available listings. - Guest: view listing detail (photos, host, reviews, live availability), then book.

**Non-functional — the crux** - **Search is the hot path:** read-heavy, sub-300ms, must combine a **geo query** + an **availability query** + **filters**, at millions-of-listings scale. - **Small staleness in search is acceptable** — re-validated authoritatively at booking. - **High availability for search** — that's where the revenue funnel starts. - **Global:** listings and guests are worldwide → data locality for latency and GDPR.

**Candidate:** The sentence I'd say out loud early: **search needs to be fast, relevant, and "good enough" fresh over a geo+date+filter query at huge scale; booking needs to be exactly right over one listing and a handful of nights.** Different problems, different storage, connected by one re-validation step.

---

## 2. Estimation

7M active listings worldwide. 150M searches/day, 2M bookings/day.

search QPS =  $150,000,000 / 86,400 \approx 1,700$  QPS avg (peak 6–10x → ~12k QPS)

booking QPS =  $2,000,000 / 86,400 \approx 23$  QPS avg

→ read:write ratio ~ 1000:1. Booking traffic is a rounding error next to search.

A dragged map viewport can span thousands of candidate listings before date/filter narrowing → need "thousands of geo candidates" → "top 20 ranked" in one fast query.

Storage:

|                                                              |                                                              |             |                        |
|--------------------------------------------------------------|--------------------------------------------------------------|-------------|------------------------|
| listing metadata                                             | : 7M * ~5 KB                                                 | = 35 GB     | → SQL, trivial         |
| availability calendar                                        | : 7M * 365 nights * 20B                                      | = ~50 GB/yr | → SQL, trivial         |
| photos                                                       | : 7M * 20 * 200KB                                            | = ~28 TB    | → object storage + CDN |
| search index (denormalized doc per listing, geo-partitioned) | = few hundred GB → fits a sharded search cluster comfortably |             |                        |

**Candidate:** The number that drives the design: **~1,700 avg / ~12k peak QPS of geo+date+filter reads over 7M listings, sub-300ms** — a relational `WHERE lat BETWEEN...` scan does not survive that. I'll design around it, then attach the booking handoff.

## 3. V1 — the naive shape (and why it breaks)

```
[App] --GET /search?bounds&dates&filters--> [Server] --> [Postgres]
                                SELECT * FROM listings
                                JOIN availability ON ...
                                WHERE lat BETWEEN :a AND :b
                                   AND lng BETWEEN :c AND :d
                                   AND price BETWEEN ...
                                ORDER BY relevance
```

**Why it dies:** a `lat/lng BETWEEN` scan can't do a true 2D range efficiently — Postgres picks one index, scans a wide slice, filters the rest in memory. Bolting a 365-row availability join and filters onto that multiplies the cost. At 7M rows / 12k QPS this is a near-full-table scan repeated thousands of times a second, and SQL isn't built to relevance-score millions of candidates either.

**Candidate:** I need (1) a purpose-built geo index, and (2) a pre-computed "is this listing generally free" signal so the hot query isn't joining against 365-row calendars live.

## 4. Geo-indexing — geohash, and why a search engine, not SQL

**Interviewer:** How do you actually make "listings inside this rectangle" fast?

**Candidate: Geohash** — encode `(lat, lng)` into a base-32 string where nearby points share prefixes. A 6-char cell covers roughly 1.2km x 0.6km. Bucket listings by geohash cell; "search this viewport"

becomes "which cells intersect this rectangle," a handful of prefix lookups instead of scanning every listing on Earth.

GEOHASH GRID (~1km cells)

lng ->

lat +-----+-----+-----+-----+

| | 9q8v | 9q8y | 9q8z | 9q9n |

v +-----+-----+-----+-----+

| 9q8t | 9q8w | 9q8x | 9q9j | <- viewport straddles these 2 cells

+-----+-----+-----+-----+

| 9q8m | 9q8q | 9q8r | 9q9h |

+-----+-----+-----+-----+

viewport -> covering cells [9q8w, 9q8x] -> fetch listings bucketed under those prefixes -> exact bbox + date + amenity filters on that small set

A **quadtree** is the alternative: recursively split the map into quadrants until each leaf has a manageable listing count — denser cities get finer cells automatically. Either way, the point is turning "point inside an arbitrary rectangle" into "a few pre-computed cells."

**Why a search engine (Elasticsearch/OpenSearch) and not SQL's spatial index (PostGIS R-tree):** I

don't just need geo — I need geo **AND** filters **AND** price range **AND** a relevance score, combined, at 12k QPS, sharded and replicated. That's exactly what a distributed search engine is built for: native `geo_point` / `geo_bounding_box` queries, inverted indexes for filters, and a scoring pipeline, all horizontally scalable.

## 5. Availability — modeling and querying it without a live join

**Interviewer:** Now cut the geo candidates down to "available for these 5 nights," without a 365-row join per listing on the hot path.

**Candidate:** Two layers — **source of truth** for correctness, **pre-rolled summary** for speed.

**Source of truth (SQL), one row per listing per night:**

```
Table: availability
listing_id (FK), date, is_available (BOOL), price_override
PRIMARY KEY (listing_id, date)
```

```
[OPEN] --host blocks / guest books--> [BLOCKED] --unblock/cancel--> [OPEN]
```

A 5-night stay = 5 contiguous rows that must ALL be OPEN; booking flips them atomically (row-locking – problem #6's territory, not re-derived here).

Exact and cheap to write, but checking it for thousands of geo candidates per search would blow the latency budget with point reads.

**Fast path — denormalized into the search index:** on every availability change, an async job recomputes a compact "available date ranges" summary and pushes it into the listing's search document. Search then intersects candidate ranges against it directly, in the same query that already returned geo+filter matches.

```

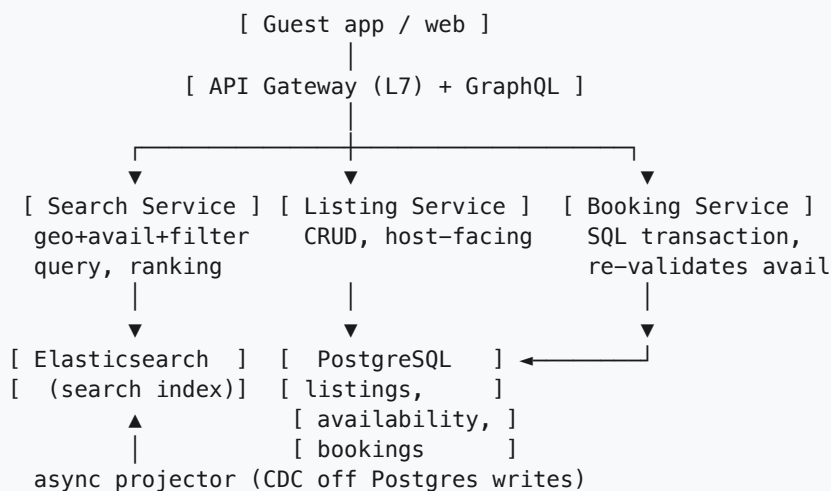
SQL: listing 42: 08-01..08-03 OPEN, 08-04 BLOCKED, 08-05..08-10 OPEN
    | async projector (on every availability write)
    ↓
Search doc: { listing_id:42, geo:{...}, avail_ranges:[[08-01,08-03],[08-05,08-10]] }
    |
    ↓
Query "check-in 08-05, check-out 08-08" -> intersects avail_ranges -> match

```

**Why not query SQL for availability on the hot path anyway?** The hot path already hits the search engine for geo+filters+ranking — forcing a per-candidate SQL round-trip too turns one query into thousands of sequential lookups. Pre-rolling availability into the same document keeps it one query, one system, one round trip.

**The tradeoff, said out loud:** the index's availability view can lag by seconds — search is a *hint*. **The booking service always re-checks the real `availability` rows in SQL, transactionally, before confirming.** Worst case a guest sees "no longer available" and retries — annoying, rare, never a real double-booking.

## 6. V1 architecture



Postgres is the source of truth (booking needs multi-row ACID); Elasticsearch is a denormalized read index rebuilt asynchronously — never written to directly by the app. Search never blocks on booking's consistency needs, and booking never depends on search's speed.

## 7. Scaling — 12k QPS peak, worldwide

**Interviewer:** It's Saturday morning in every timezone at once, 12k QPS on search globally. What changes?

**Candidate:** Cheapest-first:

1. **Cache repeated queries.** Thousands of guests search roughly the same viewport + dates ("Paris, this weekend"). Cache the rendered result page in Redis, keyed by (geohash-cells, date-range, filter-hash), TTL 30–60s — a large share of 12k QPS collapses to cache hits.
2. **Shard Elasticsearch by region** so "Paris" queries only touch Paris shards; add read replicas per shard so throughput scales with replica count.
3. **Autoscale the stateless search-service tier** — query construction, cache lookups, result shaping; no state, scales horizontally trivially.

Write volume (host edits) is tiny by comparison; the one guard needed is rate-limiting the async projector so a bulk price update across 50,000 listings doesn't starve normal reindexing.

---

## 8. Multi-region — data locality, not just replicas

---

**Interviewer:** A guest in São Paulo searches Lisbon while someone in Tokyo searches Osaka, simultaneously. How do you lay this out across regions?

**Candidate:** Route by the geography being searched, not by where the request originated — listings are inherently geo-located.

```
[Guest in São Paulo] -> GeoDNS -> nearest gateway
    "search Lisbon" -> routed to EU region's Elasticsearch shards
                        that OWN Lisbon's listings (data locality)
    booking write for a Lisbon listing -> Postgres primary that owns
                                        that listing (its home region)
```

- Search shards by **listing** region: a São Paulo guest searching Lisbon pays one cross-ocean hop — rare, since most guests search near themselves.
- Postgres is partitioned by region too, which also buys **GDPR/data residency** — EU host/guest PII stays inside EU infrastructure.
- Photos/CDN are already global by nature, independent of this geo-partitioning.

---

## 9. Latency budget

---

|                                 |                       |                                                              |
|---------------------------------|-----------------------|--------------------------------------------------------------|
| Client -> Edge/Gateway (GeoDNS) | ~10–30ms              |                                                              |
| Redis cache check               | ~1–2ms                | -- HIT ends here (~15–35ms total)                            |
| MISS ↓                          |                       |                                                              |
| Search Service -> Elasticsearch | ~20–60ms              | geo bbox + avail + filter + relevance, sharded fan-out+merge |
| Result shaping                  | ~5–10ms               |                                                              |
| -----                           |                       |                                                              |
| Cache HIT: ~15–35ms             | Cache MISS: ~40–100ms | (inside the 300ms budget)                                    |

To cut further: route each query to *only* the shards intersecting the viewport (never scatter-gather every shard), and pre-warm the cache for popular cities/weekends instead of waiting for the first cold miss.

---

## 10. Consistency — what's strong, what's eventual

- **Strong:** `availability` and `bookings` in Postgres — a booking transaction flips N night-rows atomically. Not negotiable.
- **Eventual:** everything in Elasticsearch — geo index, availability summary, cached price; can lag Postgres by seconds. A guest might see a listing as available moments after someone else booked it.
- **Why that's fine:** search's job is a *good candidate list fast*, not the final decision. **Booking re-validates against SQL before confirming** — staleness in search can only ever cause a "no longer available" retry, never an actual double-booking. That re-validation seam is why the two stores can run different consistency models safely.

## 11. Async — what moves off the critical path

```
Host writes (listing edit / price / calendar block)
  | Postgres commit
  ▼ CDC / outbox event
  [ Kafka ] (partitioned by listing_id, durable, replayable)
    |
    ▼
  [ Indexer workers ] -> recompute avail_ranges + geo doc -> [Elasticsearch]
                      -> invalidate Redis keys touching that listing

Booking confirmed (Postgres commit)
  |
  ▼
  [ Kafka: booking-events ] -> confirmation email/SMS, host alert, analytics
```

The host's edit is durable the instant Postgres commits; reindexing happens a beat later. **Why CDC/outbox and not a direct dual-write** (Postgres then Elasticsearch inline)? A dual-write can fail halfway — Postgres commits, ES write times out — and silently desyncs the stores forever. CDC off the write-ahead log guarantees the index reflects what actually committed, and Kafka retention lets a crashed indexer replay and catch up.

## 12. Caching — what, where, invalidation

| Layer                  | What's cached                                               | TTL / invalidation                                                                           |
|------------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| CDN (edge)             | Listing photos, static assets                               | Long TTL, content-hashed URLs — never invalidated, just re-uploaded under a new URL          |
| Redis — search results | Rendered page keyed by (geo-cells, date-range, filter-hash) | Short TTL (30-60s) — availability shifts constantly; expire rather than precisely invalidate |
| Redis — listing detail | Cache-aside: miss → Postgres+reviews → cache                | Explicit invalidation on host edit                                                           |

| Layer         | What's cached                                   | TTL / invalidation                                                    |
|---------------|-------------------------------------------------|-----------------------------------------------------------------------|
| Elasticsearch | The whole index is itself a "cache" of Postgres | Kept in sync via CDC pipeline; full reindex job for disaster recovery |

Precisely invalidating every cached search page that *might* contain a changed listing would mean tracking a combinatorial set of keys per listing. A short TTL is simpler, and the staleness is already tolerated by design — cheap and correct-enough beats complex and marginally fresher.

## 13. Technology choices — what and why

| Concern                                         | Choice                                                                            | Why this, and why not the alternative                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Geo + availability + filter search</b>       | <b>Elasticsearch/OpenSearch</b> , region-sharded, docs denormalized from Postgres | Native geo queries, inverted indexes for filters, relevance scoring, horizontal sharding. <i>Not Postgres/PostGIS</i> — geo + filters + numeric range + custom scoring at 12k QPS across 7M rows degrades fast when that many predicates stack on one relational query; search engines are built for exactly this OLAP-ish denormalized shape.                                          |
| <b>Listings, availability, bookings (truth)</b> | <b>Relational DB (PostgreSQL)</b>                                                 | Booking a date range needs a multi-row ACID transaction (flip N night-rows atomically); availability-per-night is a natural ( <code>listing_id</code> , <code>date</code> ) PK with row locking. <i>Not a NoSQL/KV store</i> — you'd lose multi-row transactional guarantees exactly where correctness matters most and hand-roll what SQL gives free.                                  |
| <b>Search index ↔ SQL sync</b>                  | <b>CDC / transactional outbox → Kafka → async indexer</b>                         | Index reflects what actually committed; survives indexer crashes via replay. <i>Not a synchronous dual-write</i> — a mid-write failure silently desyncs the stores with no record of the gap.                                                                                                                                                                                           |
| <b>Listing detail page API</b>                  | <b>GraphQL</b>                                                                    | The screen needs listing fields + host profile + a slice of reviews + photos + an availability window, nested, shaped differently per client. One round trip avoids over-fetch (bloated REST blob) or 4-5 chained REST calls that hurt on mobile. <i>Not plain REST here</i> — REST's fixed shapes force over-fetch-or-multiple-round-trips; GraphQL asks for exactly the needed shape. |
| <b>Simple/cacheable endpoints</b>               | <b>REST</b> (e.g. <code>/listings/{id}/ical</code> , webhooks)                    | Simpler, more CDN-cache-friendly than GraphQL where flexible shaping isn't needed.                                                                                                                                                                                                                                                                                                      |

| Concern                              | Choice                                                                                    | Why this, and why not the alternative                                                                                                                                                                                                                               |
|--------------------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Result &amp; detail caching</b>   | <b>Redis</b>                                                                              | Sub-ms reads absorb most repeated searches/views; TTL expiry is an adequate answer to constantly-changing availability. <i>Not skipping caching</i> — at 12k peak QPS, re-hitting ES/Postgres for repeated queries wastes capacity needed for the unique long tail. |
| <b>Photos/video</b>                  | <b>Object storage (S3) + CDN</b>                                                          | Large immutable blobs don't belong in a row-store or search index; CDN means origin is touched once per region per photo.                                                                                                                                           |
| <b>Booking correctness mechanism</b> | <i>(out of scope — problem #6)</i> row-level locking on availability inside a transaction | This doc's job is the seam: search treats availability as a hint, booking treats it as ground truth and re-checks it.                                                                                                                                               |

**Say-it-out-loud justification:** *"Postgres is the ACID source of truth for listings, per-night availability, and bookings, because booking is a multi-row transaction that must never partially apply; Elasticsearch is a denormalized, geo-sharded search index rebuilt asynchronously off Postgres via CDC, because geo-ranking-plus-filters across millions of listings is a fundamentally different query shape than transactional lookups; Redis caches the hot repeated slice of both; and GraphQL fronts the listing-detail page because that screen needs several nested resources shaped differently per client in one round trip. The rule that keeps two different consistency models correct together: search is a hint, and booking always re-validates against Postgres before it ever confirms."*

## 14. Failure modes & graceful degradation

| Failure                          | What happens                                      | Mitigation                                                                                                                            |
|----------------------------------|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Elasticsearch shard/cluster down | Search degraded for that region                   | Serve from replica shards; if the whole cluster is down, fall back to a SQL-backed "browse by city" (worse ranking, still functional) |
| Search index lags Postgres       | Listing shows available but was just booked       | Designed-for (§10) — booking re-checks SQL transactionally; guest re-searches, never double-booked                                    |
| Redis down                       | Cache misses fall through to ES/Postgres directly | Slower, still correct                                                                                                                 |
| Postgres primary down (a region) | Writes in that region pause briefly               | Promote a replica; search/browsing unaffected (doesn't hit primary)                                                                   |
| CDC/indexer pipeline backs up    | Search freshness lag grows                        | Kafka retains events; indexer catches up; worst case is a longer hint window, never wrong data                                        |
| CDN/object storage blip          | Photo fails to load                               | Placeholder image; never fail the whole listing page for one asset                                                                    |



## 15. Follow-up curveballs

---

**Interviewer:** How do you support "search as I drag the map," continuously?

**Candidate:** Debounce client-side (fire on drag-pause or a distance threshold, not every pixel), then issue a `geo_bounding_box` query for the new viewport. Since results are cached by geohash-cell set, a small nudge overlapping mostly-cached cells is cheap; a big jump (new city) is a cold query like any other.

**Interviewer:** How do you rank results, not just filter them?

**Candidate:** A scoring layer atop the Elasticsearch query — blend geo-distance, price, rating, past booking-conversion, host responsiveness into one relevance score (a function-score query, or a learned model reordering the top-K). Availability and geo *filter*; ranking decides the *order* among what's left.

**Interviewer:** A Tokyo host blocks a date the instant a São Paulo guest is mid-checkout on that same night. What happens?

**Candidate:** Same guard as any double-booking prevention: only the booking service's transaction can flip an `availability` row, via row-level lock/conditional update. Whoever's write reaches Postgres first wins; the other gets "no longer available" — deterministic, one row, one transaction. Search never participates in this decision.

**Interviewer:** Marketing wants a "12 people are looking at this listing" badge.

**Candidate:** A cosmetic, high-write counter that must never contend with anything correctness-critical — a Redis counter with short expiry per listing, incremented on page view, read approximately. Same pattern as any display counter: isolate it entirely from the availability/booking keys.

---

## 16. Deep-dive: "Why not just one SQL query — join listings, availability, and a geo filter?"

---

**Interviewer:** I keep coming back to this. Why not skip Elasticsearch entirely? One query: `SELECT ... FROM listings JOIN availability ... WHERE lat/lng BETWEEN ... AND amenities @> ...`. Postgres can index all of that. Why is that wrong?

**Candidate:** It's the right question, because on paper every ingredient — geo columns, a date table, filter columns — is individually indexable in Postgres. The reason it still fails isn't any single predicate; it's that the query needs **all of them to combine, per request, at 12k QPS**, and each dimension multiplies the cost of the others instead of narrowing it.

**Walk the combinatorics with real numbers.** A dragged-map viewport over a city might geo-match ~5,000 listings before anything else is applied. Now stack the rest:

```
5,000 geo candidates
x 5 nights per stay (need ALL 5 nights OPEN, contiguous)
= 25,000 availability-row reads, per single search request
x amenity/price filters on top (each listing re-checked against N predicates)
x 12,000 requests/sec at peak
-----
~300 MILLION availability-row touches/sec just from this one screen,
before Postgres even ranks anything.
```

That's not "a slow query" — it's a fundamentally different order of work than what a transactional row-store is built to amortize. A few concrete reasons it doesn't just get slower gracefully, it falls over:

- **The date-range check isn't a simple WHERE .** "Available for all 5 nights" is really `COUNT(available nights in range) = 5` per listing — an aggregation joined against a 365-row-per-listing table, computed *per candidate, per request*. SQL can express it, but the planner has to actually do that aggregation live every time; nothing about it is cacheable across different date ranges.
- **The geo predicate alone isn't selective enough to save you.** A `lat/lng BETWEEN` range scan (no true 2D index without PostGIS, and even PostGIS's R-tree isn't built to also rank by relevance) still returns thousands of rows before the availability join even starts — so the expensive part happens on a candidate set that hasn't been narrowed by date yet.
- **No caching layer helps a live join.** Query-result caching works when many requests share a cache key. Here, viewport + dates + filters is a huge combinatorial key space — "Paris, Aug 5–8, pool+wifi" and "Paris, Aug 6–9, pool+wifi" are different cache entries that share almost all their underlying data, but a raw SQL join can't exploit that overlap; a precomputed index document can, because availability is already flattened into ranges *once*, asynchronously, off the write path.
- **Relevance ranking doesn't exist in this model.** Even if the join returned fast, you'd still need a second pass to score 5,000 rows by distance/price/rating/conversion — work a search engine does natively as part of the same query.

**The fix isn't a better SQL query — it's not doing this computation live at all.** Precompute the availability *shape* (open date-ranges, not raw nights) into each listing's search document asynchronously, whenever the underlying row changes, so the hot path is a single index lookup that already has geo + filters + availability + ranking baked in — zero live joins, zero live aggregation.

|                  | Live SQL join (listings ⋈ availability ⋈ geo filter)                                               | Denormalized search index (Elasticsearch, availability projected in)                                 |
|------------------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Per-request cost | Recomputes geo scan + N-night aggregation + filter predicates from scratch, every request          | One index lookup against pre-flattened <code>geo + avail_ranges + filters</code> doc                 |
| Date-range check | Live <code>COUNT(nights)=N</code> aggregation per candidate, per request                           | Pre-rolled into <code>avail_ranges</code> at write time; query does a cheap range-intersection       |
| Geo query        | <code>lat/lng BETWEEN</code> scan (or PostGIS R-tree) — selective, but not combinable with scoring | Native <code>geo_bounding_box / geo_distance</code> , combinable with filters and score in one query |

|                                         | Live SQL join (listings $\bowtie$ availability $\bowtie$ geo filter)                                                 | Denormalized search index (Elasticsearch, availability projected in)                                             |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Relevance ranking</b>                | Not native — a separate pass over the result set                                                                     | Native function-score / ML-reorder as part of the same query                                                     |
| <b>Scales with QPS?</b>                 | No — every request repeats the full join; throughput caps around the join's cost, not the hardware                   | Yes — horizontally sharded, replicated read nodes; query cost is ~constant per shard regardless of QPS           |
| <b>Freshness</b>                        | Perfectly fresh (reads the live table)                                                                               | Seconds of lag (async projector/CDC)                                                                             |
| <b>Where it's the right call anyway</b> | Host's own listing detail page, admin tooling, low-QPS internal queries — anywhere freshness matters more than scale | The public search/browse hot path — anywhere scale and combined predicates matter more than sub-second freshness |

**Rule of thumb:** *A live relational join is the right tool when one predicate dominates and freshness must be perfect. Once you're combining several independent, high-cardinality predicates (geo  $\times$  date-range  $\times$  filters  $\times$  ranking) at high QPS, the fix is almost never a smarter query — it's precomputing the answer's shape asynchronously so the hot path degenerates into a single lookup.*

## 17. Deep-dive: the search index goes stale — a listing gets booked but still shows "available"

**Interviewer:** Walk me through the actual failure, second by second. A listing gets booked. For how long does search still show it as free, and what happens to the guest who tries to book it in that window? And what about the opposite — a booking gets cancelled, does the listing disappear from search for a while even though it's free again?

**Candidate:** This is the direct consequence of the design choice in §16 and §10 — search is a hint, not the truth — so let's be precise about the actual window where that hint is wrong, and what happens on both sides of it.

## The forward case: booked, but search doesn't know yet

T+0ms      Guest A's booking **COMMITTS** in Postgres.  
availability rows for nights [08-05..08-07], listing 42: OPEN -> BLOCKED

T+0ms      Guest B is *\*already\** looking at listing 42 in a cached search result page (Redis, TTL 30-60s) that said "available" – rendered before T+0.

T+50ms      Postgres WAL write triggers a CDC / outbox event: "listing 42 avail changed"

T+300ms      Event lands on Kafka topic (partitioned by listing\_id)

T+900ms      Indexer worker consumes it, recomputes avail\_ranges, writes to Elasticsearch

T+1200ms    Elasticsearch refresh interval exposes the updated doc to new queries

-----  
STALE WINDOW for listing 42 in search: ~1.2s (CDC pipeline) plus up to 60s more if Guest B is reading an already-cached search-results page (Redis TTL), not a fresh query.

T+5s      Guest B taps "Book" on listing 42, dates 08-05..08-07.  
Search told them it was free. Booking service does NOT trust that – it re-reads the SAME availability rows Guest A just flipped:

```
SELECT ... FROM availability
WHERE listing_id=42 AND date IN (08-05,08-06,08-07)
FOR UPDATE;
```

-> rows are BLOCKED -> transaction rejects Guest B's booking

T+5s      Guest B sees "No longer available" – annoying, but NEVER a double-booking. The stale window only ever produces a rejected attempt, never a conflicting confirmation, because confirmation only happens against Postgres, never against the search index.

## The reverse case: freed, but search still hides it

T+0ms      Guest A cancels. availability rows [08-05..08-07], listing 42: BLOCKED -> OPEN  
... same CDC lag ...

T+1.2s+    Elasticsearch still shows listing 42 as unavailable for those dates.

Guest C searches those dates -> listing 42 doesn't appear in results at all.  
No error, no retry, nothing for Guest C to "hit" – it's a silent false negative:  
a listing that's actually bookable simply doesn't show up for a few seconds.

The forward case is self-correcting (the authoritative re-check catches it, at the cost of one bad UX moment). The reverse case is *worse in one specific way*: there's no re-check to trigger, because the guest never sees the listing to attempt a booking on it – it's a quiet lost booking / lost revenue, not a visible error, and it never surfaces unless someone is specifically measuring search-vs-truth divergence.

## Ranked fixes

**Fix 1 — Search is approximate by contract; booking always re-validates.** (Already the baseline — §10.) This is the fix for the forward case's *correctness*, not its latency: it guarantees no double-booking regardless of how stale the index gets. It does nothing for the reverse case (nothing to re-validate if the listing was never shown), and does nothing to shrink the 1–60s window itself.

**Fix 2 — Shrink the CDC pipeline's own latency.** Most of the 1.2s in the walkthrough is pipeline hops, not unavoidable physics: skip micro-batching in the indexer, give the availability-change topic its own low-latency consumer group instead of sharing one with bulk listing-metadata reindexing, and lower the Elasticsearch refresh interval for this index (at the cost of more segment merges). This tightens the window from ~1.2s toward a few hundred ms — helps both cases, doesn't eliminate either.

**Fix 3 — Targeted cache invalidation instead of pure TTL.** Right now (§12) the Redis search-result cache just expires after 30–60s. On an availability-change event, actively invalidate the specific cached result pages that contain that listing (the projector already knows the listing's geo-cell — invalidate (that cell, overlapping date ranges) keys). This kills the *cache-layer* contribution to staleness (the 60s tail in the forward-case walkthrough), leaving only the CDC pipeline's ~1s as the residual gap.

**Fix 4 — UX signal: "confirm at checkout."** For the forward case specifically, show a lightweight "final availability confirmed at checkout" note on the booking screen rather than presenting availability as a hard guarantee. This doesn't reduce staleness, it manages guest expectations so a rare rejection reads as normal, not broken.

**Fix 5 — Priority reindex on cancellation (targets the reverse case specifically).** Cancellations are much lower volume than bookings, so it's cheap to give cancellation events a dedicated high-priority queue that jumps ahead of the routine reindex backlog — since a freed listing not showing up is pure lost revenue with no self-correcting mechanism, it's worth spending disproportionate pipeline priority on shrinking *this* side of the window specifically, even if the booked-side window stays a bit longer.

**Recommended combination:** keep Fix 1 as the non-negotiable correctness backstop, add Fix 2 (fast dedicated CDC lane for availability, separate from bulk reindexing) to shrink the window itself, add Fix 3 (targeted invalidation) so the Redis cache layer stops being the long pole, and add Fix 5 to specifically protect against silent lost bookings on cancellation. Fix 4 is cheap and worth doing regardless, but it's a UX patch, not a fix for the underlying lag.

**What to say out loud:** *"The forward case — booked but still shown available — is already handled by design: search is a hint, booking re-validates against Postgres before confirming, so it can only produce a rejected attempt, never a double-booking. The reverse case — freed but still hidden — is the sharper problem, because there's no re-check to catch it; it's a silent, unmeasured revenue leak. I'd shrink the CDC pipeline's latency with a dedicated low-latency lane for availability events, add targeted cache invalidation so the Redis layer stops adding its own tail, and specifically prioritize cancellation events in that pipeline, since that's the side of the staleness window that doesn't self-correct."*

---

## Summary — the mental model

---

Airbnb's hard problem is **finding, not booking**: a geo + date-availability + filter query over millions of listings, ranked, in under 300ms, while the actual booking transaction is comparatively tiny and rare (1000:1 read:write). The winning shape: **Postgres as the ACID source of truth** for listings, availability, and bookings; **Elasticsearch as a geo-sharded, denormalized search index** kept eventually consistent via CDC, because geo+filter+ranking is a different query shape than transactional lookups; **geohash-style spatial indexing** to turn "point in a rectangle" into a few cell lookups; a **rolled-up availability**

**summary** projected into the same search document so the hot path never live-joins per listing; **Redis caching** on the predictably-repeated slice of both; and **GraphQL** fronting the listing page because it's inherently a nested, client-shaped fetch. The rule holding the two consistency models together: **search is a hint, booking is the truth, and booking always re-validates before it confirms.**